Name: _____
Surname: _____
Date: _____

Multiple Choice (10 points each)

Instructions

Circle the correct answer for each question, as each question has **only one** correct answer. Provide reasoning for your answer in the space next to each question.
A correct answer always awards full points (+10 pts).
An incorrect answer with sensible reasoning awards half points (+5 pts).
An incorrect answer with faulty or no reasoning subtracts points (-3 pts).
No answer gives zero points (neither adds nor subtracts points).
Assume each code cell is **run in isolation**, meaning no other code has been run before it.

Questions

1. You want to compute the following expression in Python: $(a + b^2)/(2c)$. Which one of the following options does it right?

   (a) `(a + b^2) / (2 * c)`

   (b) `(a + b**2) / (c * 2)`

   (c) `(a + b**2) / 2 * c`

2. After running the following code, what will be the final value of variable "s"?

```
s = "aBcDeFg"
s = s[1::3].lower()
```

   (a) `s = "adg"`

   (b) `s = "be"`

   (c) The code will raise an error.

3. What is the purpose of the "else" statement in an "if-else" block?

   (a) To execute code only if all the above "if" conditions are False.

   (b) To specify an alternative condition.

   (c) To indicate the end of the code block.

4. After running the following code, what will be the final value of variable "x"?

```
st_a = {1, 2, 3}
st_b = {2, 2, 3}
st_c = list(st_a.union(st_b))
x = sorted(st_c)[0]
```

   (a) `x = 1`

   (b) The code will raise an error.

   (c) We cannot predict the outcome.

5. What is the difference between "print(x)" and "return x" inside an user defined function?

   (a) "print(x)" and "return x" are interchangeable; they both output the value of "x".

   (b) "print(x)" outputs the value of "x" to the console, while "return x" sends the value of "x" back to the calling code.

   (c) "print(x)" will cause an error if we use it inside an user defined function.

6. Consider the following code. Which will be the value of "x" in the **global scope**?

```
x = 20

def exercise(y: int) -> int:
  x = 4
  return y / x

x = exercise(x)
```

   (a) `x = 4`

   (b) `x = 5`

   (c) `x = 20`

Answers: 1-b, 2-b, 3-a, 4-a, 5-b, 6-b

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

7. String Patterns (20 points)

Create a function named `count_pattern(main_str, pattern)` that takes two parameters:

- `main_str`: A string in which you have to find the pattern.
- `pattern`: A string pattern which you have to find in the `main_str`.

The function should return the number of times the `pattern` occurs in the `main_str`, including overlapping instances. For instance, the pattern `"121"` appears three times in `"1212121"`.

Answer:

The function receives two strings, `main_str` and `pattern`, and returns an integer: the number of matches, including overlapping ones. *(+3 points)*

We initialize a counter before the loop, `count = 0`, and we return it after the loop. *(+2 points)*

We need to look at every position of `main_str` where a match could start, so we use a for loop: `for idx in range(len(main_str) - len(pattern) + 1)`. The "+1" is needed to include the last possible starting position. Because we visit every position, including the ones inside a previous match, overlapping matches are counted. *(+7 points)*

Inside the loop, we take a slice of `main_str` with the same length as the pattern, `main_str[idx:idx+len` and compare it with the pattern using an if-clause. If they are equal, we add 1 to the counter. *(+8 points)*

Comment: Using `main_str.count(pattern)` is not correct, as it does not count overlapping matches: on `"1212121"` it returns 2 instead of 3. Another valid approach is to use `main_str.find(pattern, start)` inside a while loop, where each new search starts at one position after the last match (`pos + 1`).

(Optional, code)

```
def count_pattern(main_str: str, pattern: str) -> int:
    count = 0
    # Look at every possible starting position
    for idx in range(len(main_str) - len(pattern) + 1):
        # Take a slice of the same length as the pattern
        if main_str[idx:idx+len(pattern)] == pattern:
            count += 1
    return count
```

8. Pair Distance Calculator (20 points)

In 2D space, a point can be represented as a pair of coordinates (x, y). The Euclidean distance between two points (x_1, y_1) and (x_2, y_2) is given by the formula:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Your task is to implement a function `compute_distances(reference_point, point_list)` where:

- `reference_point` is a tuple of two coordinates (x, y).
- `point_list` is a list of such tuples, each representing a point in the 2D space.

The function should return a list of distances where each distance corresponds to the Euclidean distance between the `reference_point` and each point in `point_list`.

Answer:

The function receives the reference point, a tuple of two numbers, and a list of points, which is a list of tuples. It returns a list of numbers: one distance for each point, in the same order. *(+3 points)*

Before the loop, we initialize an empty list, `distances = []`, where we will store the results. After the loop finishes, we return it. *(+3 points)*

We need one distance for each point, so we use a for loop, `for point in point_list`. Each `point` is a tuple, so we can unpack its coordinates with `x2, y2 = point`. We unpack the reference point in the same way, before the loop, with `x1, y1 = reference_point`. Indexing, such as `point[0]` and `point[1]`, is equally valid. *(+5 points)*

Inside the loop, we apply the formula. The squares are computed with `**2`. As we have no square root function in the cheatsheet, we use a power of one half, `**0.5`. Be careful with the parentheses, as the exponent must affect the whole sum: `((x2 - x1)**2 + (y2 - y1)**2) ** 0.5`. Using `math.sqrt()` after `import math` is also accepted. *(+5 points)*

Finally, we add each distance to the list, using `distances.append(distance)`. *(+4 points)*

(Optional, code)

```
def compute_distances(reference_point: tuple, point_list: list[tuple]) ->
    list[float]:
    # Initialize the output list
    distances = []
    # Unpack the coordinates of the reference point
    x1, y1 = reference_point
    # Loop through the points, one at a time
    for point in point_list:
        x2, y2 = point
        # Apply the Euclidean distance formula
        distance = ((x2 - x1)**2 + (y2 - y1)**2) ** 0.5
        distances.append(distance)
    return distances
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.

Given a list or tuple named `ls`:

- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` returns the union of set `a` with another set `b`.
- `a.intersection(b)` returns the intersection of set `a` with another set `b`.
- `a.difference(b)` returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.